Artificial Intelligence

Assignment 3

Group: Ritchie

1. Eshwar Reddy Poreddy: 221100642
(eshwaraa07@uni-koblenz.de)

2. Prajna Shetty: 222100479
(prajnashetty73@uni-koblenz.de)

3. Vamshidhar Pratap: 222202552
(vamshidharpratap@uni-koblenz.de)

4. Yangsun Kim: 220202400
(ykim@uni-koblenz.de)

1 Lists in Prolog (5 Points)

Develop a prolog predicate lists intersect/2, which tests for any two lists if at least one element is contained in both lists. Hints:

- The /2 after the predicate name indicates the arity of the predicate to be developed. You are allowed to use the built-in predicate member/2.

For example, the following queries should result in answer true:

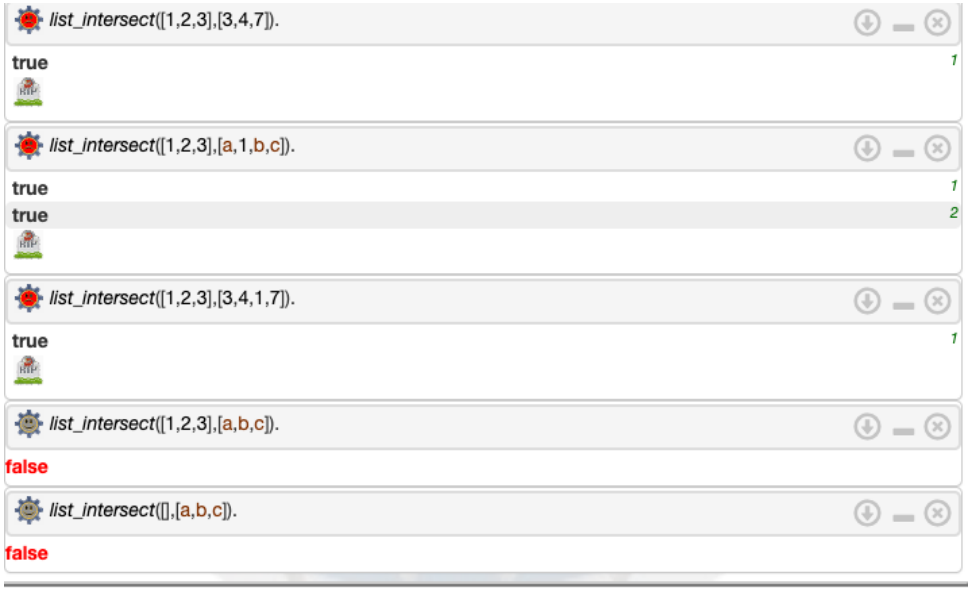
- lists intersect ([1,2,3], [3,4,7]).
- lists intersect([1,2,3], [a,1,b,c]).
- lists intersect([1,2,3], [3.4.1,7]).

And the following queries should result in answer false:

- lists intersect([1,2,3]. [a,b,c]).
- lists intersect([ ], [a,b,c)).

Answer

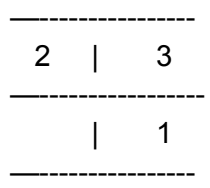
```
list_intersect(L1,L2):-
        member(X,L1),
        member(X,L2).
```



2 Search Problems

Encode the following problems as search problems as in the lecture: Define what a state is, the transition function as well as the initial state and set of goal states.

1. The 15 puzzle (https://en.wikipedia.org/wiki/15 puzzle). While the classic variant of the puzzle is played on a 4x4 board with 15 tiles, you can restrict yourselves to a 2x2 board with 3 tiles with this initial position:



2. Four gnomes G1, G2, G3, G4 are on the right side of a wobbly bridge. Its dark and the group only has one torch. The task is to get everyone to the left side. At most two gnomes can cross the bridge together. They need the torch in order to cross the bridge. The torch burns for 60 minutes. G1 needs 5 minutes to get across, G2 needs 10, G3 needs 20 and G4 needs 25 minutes. If two gnomes cross the bridge together, they need as much time as the slowest of them.

State : A state is the representation of the problems at the given moment.
Transition Function : A function that takes action and returns a new state.
Initial State : The initial state represents the starting configuration.
From the initial state all the series of actions will begin.
Set of goal state : The set of all states reachable from initial state to the goal state

For the puzzle,

P = (S, O, I, G)
S = { [[x1, x2], [x3, x4]] | { x1, x2, x3, x4}={0,1,2,3} }
O ={ ([[2,3],[0,1]]), ([[2,3],[1,0]]), ([[2,0],[1,3]]), ([[0,2],[1,3]]), ([[1,2],[0,3]]),
([[1,2],[3,0]]),..... }
I = ([[2,3],[0,1]])
G = ([[1,2],[0,3]])

For the G nomes

S = { (L, R, t) | L, R ⊆ { G1, G2, G3, G4 }, t ∈ { l, r }}
where,
‘L’ is for the G nomes on the Left side
‘R’ is for the G nomes on the Right side
‘t’ represents the Torch which is ‘r’ if it is on Right side or ‘l’ if it is on Left side

O = { (({}, {G1,G2,G3,G4 }, r), ({ G1,G2}, {G3,G4, }, l)), (({G2}, {G1,G3,G4, }, r),
({ G2,G3,G4}, {G1, }, l)), (({ G3,G4}, {G1,G2 }, r), ({ G1,G2,G3,G4}, { }, l)))..... }
I = ({ }, { G1,G2,G3,G4}, r)
G = ({ G1,G2,G3,G4}, { }, l)

3. Search in Prolog (5 Points)

Implement a 2x2 version of the 15 puzzle from the previous exercise in Prolog.

```
replace([_|T], 0, X, [X|T]).
replace([H|T], I, X, [H|R]) :-
    I > 0,
    NI is I - 1,
    replace(T, NI, X, R).

% Initial state
initialState(state((2, 2), [[2, 3], [0, 1]])).

% Goal state
goalState(state((1, 2), [[1, 2], [0, 3]])).

% Transition function
move(state((X, Y), Board), state((NX, NY), NewBoard)) :-
    % Find the position of the empty tile
    select(0, Row, Board),
    select(Y, Row, _),

    % Calculate the new position after the move
    NX is X + 1,
    NX =< 2,
    NY = Y,
    % Perform the move
    replace(Row, Y, 0, NewRow),
```

```
    replace(Board, X, NewRow, NewBoard).

move(state((X, Y), Board), state((NX, NY), NewBoard)) :-
    % Find the position of the empty tile
    select(0, Row, Board),
    select(Y, Row, _),

    % Calculate the new position after the move
    NX is X - 1,
    NX >= 1,
    NY = Y,
    % Perform the move
    replace(Row, Y, 0, NewRow),
    replace(Board, X, NewRow, NewBoard).

move(state((X, Y), Board), state((NX, NY), NewBoard)) :-
    % Find the position of the empty tile
    select(Row, Board),
    select(0, Row, _),

    % Calculate the new position after the move
    NY is Y + 1,
    NY =< 2,
    NX = X,
    % Perform the move
    replace(Row, Y, 0, NewRow),
    replace(Board, X, NewRow, NewBoard).

move(state((X, Y), Board), state((NX, NY), NewBoard)) :-
    % Find the position of the empty tile
    select(Row, Board),
    select(0, Row, _),

    % Calculate the new position after the move
    NY is Y - 1,
    NY >= 1,
    NX = X,
    % Perform the move
    replace(Row, Y, 0, NewRow),
    replace(Board, X, NewRow, NewBoard).
```

Bonus task (no points): Use Prolog to verify that the target position is reachable

from 2 3
- 1

Hint: You will notice quickly that a naïve implementation does not terminate. You may use any built-in predicates that might help you, e.g. length/2 modeling the length of a list, for example length([X,Y],2) is true.

```
% Define reachable state
reachable(State, []) :- goal_state(State).
reachable(State, [Move|Moves]) :-
  move(State, Move),
  apply_move(State, Move, NewState),
  reachable(NewState, Moves).

% Define applying move
apply_move([[A,B],[C,D]], [[X,Y],[Z,W]], [[X,B],[Z,D]]).

% Define a predicate to check if the goal state is reachable from the initial state
is_reachable :-
  initial_state(InitialState),
  reachable(InitialState, Moves),
  writeln('Moves to reach the goal state:'),
  writeln(Moves).
```

??????